

Entropiq — Руководство по развёртыванию

Персональный финансовый помощник с учётом инфляции

Версия документа: 1.0

Дата: 2026-02-08

Содержание

1. [Пререквизиты](#)
2. [Локальное окружение \(разработка\)](#)
3. [Stage-окружение \(тестирование\)](#)
4. [Production-окружение](#)
5. [CI/CD — GitHub Actions](#)
6. [CD — Laravel Envoy](#)
7. [Полезные команды](#)

1. Пререквизиты

1.1. Локальная машина разработчика

Инструмент	Минимальная версия	Проверка
Docker Desktop	24.x	<code>docker --version</code>
Docker Compose	2.x (v2 plugin)	<code>docker compose version</code>
Git	2.30+	<code>git --version</code>
curl	любая	<code>curl --version</code>

Важно: PHP, Composer, Node.js и npm **не нужны** на хост-машине — всё выполняется внутри Sail-контейнера. Однако при желании запускать Composer/Artisan/npm локально потребуются актуальные версии (PHP 8.3, Node 20+).

1.2. Рекомендуемые инструменты

Инструмент	Назначение	Установка (macOS)
GitHub CLI (gh)	Работа с GitHub из терминала	<code>brew install gh</code>
NVM	Управление версиями Node.js	<code>brew install nvm</code>

1.3. Stage/Production сервер

Компонент	Требования
ОС	Ubuntu 22.04 / 24.04 LTS
Docker	24.x+
Docker Compose	2.x+ (v2 plugin)
RAM	2 ГБ минимум (Timeweb VPS-2)
Disk	20 ГБ SSD минимум
Git	2.30+
Certbot	Для SSL (Let's Encrypt)

2. Локальное окружение (разработка)

2.1. Создание проекта

```
# Перейти в директорию проекта
cd ~/domains/hse

# Создать Laravel-проект через Sail
# (pgsql – PostgreSQL, redis, mailpit, meilisearch – опционально)
curl -s "https://laravel.build/entropiq?with=pgsql,redis,mailpit" |
bash
```

Sail-installer скачивает Laravel, устанавливает зависимости Composer внутри Docker-контейнера, настраивает `docker-compose.yml` с выбранными сервисами.

Если директория `entropiq` уже существует (наш случай — в ней лежит `docs/`), используйте альтернативный путь:

```
# Создать проект во временную директорию
cd ~/domains/hse
curl -s "https://laravel.build/entropiq-tmp?with=pgsql,redis,mailpit" |
bash

# Переместить файлы Laravel в основную директорию
# (docs/ сохранится)
cp -rn entropiq-tmp/* entropiq-tmp/. * entropiq/ 2>/dev/null
rm -rf entropiq-tmp
```

2.2. Настройка окружения

```
cd ~/domains/hse/entropiq

# Скопировать и настроить переменные окружения
cp .env.example .env
```

Отредактировать `.env` — ключевые переменные:

```
APP_NAME=Entropiq
APP_ENV=local
APP_DEBUG=true
APP_URL=http://localhost

# БД (Sail использует эти значения по умолчанию)
DB_CONNECTION=pgsql
DB_HOST=pgsql
DB_PORT=5432
DB_DATABASE=entropiq
DB_USERNAME=sail
DB_PASSWORD=password

# Redis
REDIS_HOST=redis
REDIS_PASSWORD=null
REDIS_PORT=6379

# Кэш и сессии
CACHE_STORE=redis
SESSION_DRIVER=redis

# Очереди
QUEUE_CONNECTION=redis

# Почта (Mailpit)
MAIL_MAILER=smtp
MAIL_HOST=mailpit
```

```
MAIL_PORT=1025
MAIL_USERNAME=null
MAIL_PASSWORD=null
MAIL_FROM_ADDRESS="noreply@entropiq.local"
MAIL_FROM_NAME="${APP_NAME}"

# Язык
APP_LOCALE=ru
APP_FALLBACK_LOCALE=en
```

2.3. Запуск Sail

```
# Первый запуск (сборка образов + старт контейнеров)
./vendor/bin/sail up -d

# Создать alias для удобства (добавить в ~/.zshrc)
alias sail='./vendor/bin/sail'

# После добавления alias:
sail up -d
```

2.4. Первоначальная настройка проекта

```
# Генерация ключа приложения
sail artisan key:generate

# Запуск миграций
sail artisan migrate

# Установка frontend-зависимостей
sail npm install

# Сборка фронтенда (dev-режим с hot reload)
sail npm run dev
```

2.5. Установка дополнительных пакетов

Устанавливать последовательно, по мере необходимости:

```
# === Обязательные пакеты ===

# Sanctum (аутентификация) – встроен в Laravel 11, публикуем конфиг
sail artisan install:api

# Livewire
sail composer require livewire/livewire livewire/volt

# Flux (UI-компоненты) – проверить актуальную команду установки на
fluxui.dev
sail composer require livewire/flux

# Horizon (мониторинг очередей)
sail composer require laravel/horizon
sail artisan horizon:install

# Telescope (отладка) – только для local/stage
sail composer require laravel/telescope --dev
sail artisan telescope:install

# Pulse (мониторинг приложения)
sail composer require laravel/pulse
sail artisan vendor:publish --
provider="Laravel\Pulse\PulseServiceProvider"

# Spatie Activity Log
sail composer require spatie/laravel-activitylog
sail artisan vendor:publish --
provider="Spatie\Activitylog\ActivitylogServiceProvider" --
tag="activitylog-migrations"

# Saloon (API-интеграции)
```

```
sail composer require saloonphp/saloon saloonphp/laravel-plugin

# Laravel Excel
sail composer require maatwebsite/excel

# DomPDF
sail composer require barryvdh/laravel-dompdf

# === Dev-зависимости ===

# Pest (тестирование)
sail composer require pestphp/pest pestphp/pest-plugin-laravel --dev -
-with-all-dependencies
sail artisan pest:install

# Larastan (статический анализ)
sail composer require larastan/larastan --dev

# Laravel Dusk (браузерные тесты)
sail composer require laravel/dusk --dev
sail artisan dusk:install

# Laravel Pint уже включён в Laravel 11
```

2.6. Конфигурация Larastan

Создать файл `phpstan.neon` в корне проекта:

```
includes:
  - vendor/larastan/larastan/extension.neon

parameters:
  paths:
    - app/

  level: 6

  ignoreErrors:

  excludePaths:
```

2.7. Конфигурация Pint

Файл `pint.json` в корне проекта (при необходимости кастомизации):

```
{
  "preset": "laravel",
  "rules": {
    "simplified_null_return": true,
    "ordered_imports": {
      "sort_algorithm": "alpha"
    }
  }
}
```

2.8. Доступные URL (локальное окружение)

Сервис	URL
Приложение	http://localhost
Mailpit (UI)	http://localhost:8025
Horizon	http://localhost/horizon
Telescope	http://localhost/telescope
Pulse	http://localhost/pulse
PostgreSQL	localhost:5432
Redis	localhost:6379

2.9. Повседневная работа

Запуск контейнеров

```
sail up -d
```

Остановка контейнеров

```
sail down
```

Запуск Horizon (очереди) – в отдельном терминале

```
sail artisan horizon
```

Запуск Scheduler (планировщик) – в отдельном терминале

```
sail artisan schedule:work
```

Запуск тестов

```
sail pest
```

Статический анализ

```
sail php ./vendor/bin/phpstan analyse
```

Проверка код-стайла

```
sail php ./vendor/bin/pint --test
```

Исправление код-стайла

```
sail php ./vendor/bin/pint

# Запуск миграций
sail artisan migrate

# Откат миграций
sail artisan migrate:rollback

# Пересоздание БД (все данные будут удалены)
sail artisan migrate:fresh --seed

# Просмотр логов контейнера
sail logs -f

# Shell внутри контейнера
sail shell

# Artisan tinker (REPL)
sail artisan tinker
```

3. Stage-окружение (тестирование)

3.1. Подготовка сервера (Timeweb VPS)

Выполнить на VPS однократно:

```
# Обновить систему
sudo apt update && sudo apt upgrade -y

# Установить Docker (официальный метод)
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
rm get-docker.sh
```

```
# Добавить текущего пользователя в группу docker
sudo usermod -aG docker $USER

# Перелогиниться для применения

# Установить Docker Compose plugin (если не установлен с Docker)
sudo apt install docker-compose-plugin -y

# Установить Git
sudo apt install git -y

# Установить Certbot (для SSL)
sudo apt install certbot -y

# Создать директорию проекта
mkdir -p /var/www/entropiq-stage
```

3.2. SSH-ключи и доступ к GitHub

```
# На VPS: сгенерировать SSH-ключ для GitHub
ssh-keygen -t ed25519 -C "entropiq-stage-deploy"

# Вывести публичный ключ
cat ~/.ssh/id_ed25519.pub

# Добавить этот ключ в GitHub → Settings → Deploy keys (read-only)
```

3.3. Клонирование репозитория

```
cd /var/www
git clone git@github.com:<username>/entropiq.git entropiq-stage
cd entropiq-stage
git checkout stage
```

3.4. Файлы Docker для Stage

3.4.1. docker/php/Dockerfile.stage

```
FROM php:8.3-fpm-alpine

# Системные зависимости
RUN apk add --no-cache \
    postgresql-dev \
    libzip-dev \
    libpng-dev \
    libjpeg-turbo-dev \
    freetype-dev \
    icu-dev \
    oniguruma-dev \
    linux-headers \
    supervisor \
    nginx \
    curl

# PHP-расширения
RUN docker-php-ext-configure gd --with-freetype --with-jpeg \
    && docker-php-ext-install \
        pdo_pgsql \
        pgsql \
        zip \
        gd \
        intl \
        mbstring \
        bcmath \
        opcache \
        pcntl

# Redis extension
RUN apk add --no-cache --virtual .build-deps $PHPIZE_DEPS \
    && pecl install redis \
    && docker-php-ext-enable redis \
    && apk del .build-deps
```

Composer

```
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
```

PHP config

```
COPY docker/php/conf.d/php-stage.ini  
/usr/local/etc/php/conf.d/custom.ini
```

Nginx config

```
COPY docker/nginx/stage.conf /etc/nginx/http.d/default.conf
```

Supervisor config

```
COPY docker/supervisor/stage.conf  
/etc/supervisor/conf.d/supervisord.conf
```

```
WORKDIR /var/www/html
```

Копирование исходного кода

```
COPY . /var/www/html
```

Установка зависимостей

```
RUN composer install --no-dev --optimize-autoloader --no-interaction
```

Frontend build

```
RUN apk add --no-cache nodejs npm \  
    && npm ci \  
    && npm run build \  
    && apk del nodejs npm \  
    && rm -rf node_modules
```

Права доступа

```
RUN chown -R www-data:www-data /var/www/html/storage  
/var/www/html/bootstrap/cache
```

```
EXPOSE 80
```

```
CMD ["/usr/bin/supervisord", "-c",  
"/etc/supervisor/conf.d/supervisord.conf"]
```

3.4.2. docker/nginx/stage.conf

```
server {  
    listen 80;  
    server_name _;  
    root /var/www/html/public;  
    index index.php;  
  
    client_max_body_size 20M;  
  
    location / {  
        try_files $uri $uri/ /index.php?$query_string;  
    }  
  
    location ~ \.php$ {  
        fastcgi_pass 127.0.0.1:9000;  
        fastcgi_param SCRIPT_FILENAME  
$realpath_root$fastcgi_script_name;  
        include fastcgi_params;  
        fastcgi_hide_header X-Powered-By;  
    }  
  
    location ~ /\.(!well-known).* {  
        deny all;  
    }  
}
```

3.4.3. docker/php/conf.d/php-stage.ini

```
; Opcache  
opcache.enable=1  
opcache.memory_consumption=128
```

```
opcache.interned_strings_buffer=16
opcache.max_accelerated_files=10000
opcache.validate_timestamps=0

; Limits
upload_max_filesize = 20M
post_max_size = 25M
memory_limit = 256M
max_execution_time = 60

; Timezone
date.timezone = Europe/Moscow

; Error handling
display_errors = Off
log_errors = On
error_log = /var/www/html/storage/logs/php-errors.log
```

3.4.4. docker/supervisor/stage.conf

```
[supervisord]
nodaemon=true
user=root
logfile=/var/log/supervisor/supervisord.log

[program:php-fpm]
command=php-fpm
autostart=true
autorestart=true
stdout_logfile=/dev/stdout
stdout_logfile_maxbytes=0
stderr_logfile=/dev/stderr
stderr_logfile_maxbytes=0

[program:nginx]
command=nginx -g "daemon off;"
```

```
autostart=true
autorestart=true
stdout_logfile=/dev/stdout
stdout_logfile_maxbytes=0
stderr_logfile=/dev/stderr
stderr_logfile_maxbytes=0

[program:horizon]
command=php /var/www/html/artisan horizon
user=www-data
autostart=true
autorestart=true
redirect_stderr=true
stdout_logfile=/var/www/html/storage/logs/horizon.log
stopwaitsecs=3600

[program:scheduler]
command=sh -c "while true; do php /var/www/html/artisan schedule:run -
-verbose --no-interaction >> /var/www/html/storage/logs/scheduler.log
2>&1; sleep 60; done"
user=www-data
autostart=true
autorestart=true
redirect_stderr=true
stdout_logfile=/dev/null
```

3.4.5. docker-compose.stage.yml

```
services:
  app:
    build:
      context: .
      dockerfile: docker/php/Dockerfile.stage
    container_name: entropiq-stage-app
    restart: unless-stopped
    ports:
```



```
- "8080:80"
environment:
  - APP_ENV=staging
volumes:
  - app-storage:/var/www/html/storage
depends_on:
  pgsql:
    condition: service_healthy
  redis:
    condition: service_healthy
networks:
  - entropiq-stage
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost/up"]
  interval: 30s
  timeout: 5s
  retries: 3
  start_period: 30s
```

```
pgsql:
  image: postgres:16-alpine
  container_name: entropiq-stage-pgsql
  restart: unless-stopped
  environment:
    POSTGRES_DB: ${DB_DATABASE:-entropiq}
    POSTGRES_USER: ${DB_USERNAME:-entropiq}
    POSTGRES_PASSWORD: ${DB_PASSWORD}
  volumes:
    - pgsql-data:/var/lib/postgresql/data
  networks:
    - entropiq-stage
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${DB_USERNAME:-entropiq}"]
    interval: 10s
    timeout: 5s
    retries: 5
```

```
redis:
  image: redis:7-alpine
  container_name: entropiq-stage-redis
  restart: unless-stopped
  command: redis-server --appendonly yes --maxmemory 128mb --
maxmemory-policy allkeys-lru
  volumes:
    - redis-data:/data
  networks:
    - entropiq-stage
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 10s
    timeout: 5s
    retries: 5

volumes:
  app-storage:
  pgsql-data:
  redis-data:

networks:
  entropiq-stage:
    driver: bridge
```

3.5. Переменные окружения Stage

Создать `.env.stage` на сервере (`/var/www/entropiq-stage/.env`):

```
APP_NAME=Entropiq
APP_ENV=staging
APP_KEY=base64:... # сгенерировать: php artisan key:generate --show
APP_DEBUG=true
APP_URL=https://stage.entropiq.ru # заменить на актуальный домен
```

```
# БД
DB_CONNECTION=pgsql
DB_HOST=pgsql
DB_PORT=5432
DB_DATABASE=entropiq
DB_USERNAME=entropiq
DB_PASSWORD=<STRONG_PASSWORD_HERE>

# Redis
REDIS_HOST=redis
REDIS_PASSWORD=null
REDIS_PORT=6379

# Кэш и сессии
CACHE_STORE=redis
SESSION_DRIVER=redis

# Очереди
QUEUE_CONNECTION=redis

# Почта (Mailpit не нужен на stage – используем Mailtrap или лог)
MAIL_MAILER=log

# Telescope
TELESCOPE_ENABLED=true

# SMS Aero (тестовый режим)
SMSAERO_EMAIL=<email>
SMSAERO_API_KEY=<key>
SMSAERO_SIGN=Entropiq
SMSAERO_TEST_MODE=true

# LLM
GIGACHAT_CLIENT_ID=<client_id>
GIGACHAT_CLIENT_SECRET=<secret>
```

3.6. Запуск Stage

```
cd /var/www/entropiq-stage

# Копировать .env
cp .env.stage .env

# Собрать и запустить контейнеры
docker compose -f docker-compose.stage.yml up -d --build

# Выполнить миграции
docker compose -f docker-compose.stage.yml exec app php artisan
migrate --force

# Очистить и прогреть кэш
docker compose -f docker-compose.stage.yml exec app php artisan
config:cache
docker compose -f docker-compose.stage.yml exec app php artisan
route:cache
docker compose -f docker-compose.stage.yml exec app php artisan
view:cache
docker compose -f docker-compose.stage.yml exec app php artisan
event:cache
```

3.7. Reverse Proxy с SSL (Nginx на хосте)

На VPS установить Nginx как reverse proxy перед Docker-контейнером:

```
sudo apt install nginx -y
```

Создать конфиг /etc/nginx/sites-available/entropiq-stage :

```
server {
    listen 80;
    server_name stage.entropiq.ru; # заменить на актуальный домен
```

```
location / {
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name stage.entropiq.ru; # заменить на актуальный домен

    ssl_certificate
/etc/letsencrypt/live/stage.entropiq.ru/fullchain.pem;
    ssl_certificate_key
/etc/letsencrypt/live/stage.entropiq.ru/privkey.pem;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;
    ssl_prefer_server_ciphers on;

    client_max_body_size 20M;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

```
# Активировать конфиг
sudo ln -s /etc/nginx/sites-available/entropiq-stage /etc/nginx/sites-enabled/

# Получить SSL-сертификат (перед этим домен должен быть направлен на VPS)
sudo certbot certonly --nginx -d stage.entropiq.ru

# Проверить конфиг и перезапустить Nginx
sudo nginx -t
sudo systemctl reload nginx
```

3.8. Обновление Stage

```
cd /var/www/entropiq-stage

# Забрать последние изменения
git pull origin stage

# Пересобрать и перезапустить
docker compose -f docker-compose.stage.yml up -d --build

# Миграции
docker compose -f docker-compose.stage.yml exec app php artisan migrate --force

# Очистить кэш
docker compose -f docker-compose.stage.yml exec app php artisan config:cache
docker compose -f docker-compose.stage.yml exec app php artisan route:cache
docker compose -f docker-compose.stage.yml exec app php artisan view:cache
```

4. Production–окружение

4.1. Отличия от Stage

Параметр	Stage	Production
APP_ENV	staging	production
APP_DEBUG	true	false
Telescope	включён	отключён
Xdebug	отключён	отключён
Opcache validation	выключена	выключена
Логирование	debug	error
Порт Docker	8080	80
VPS	общий	выделенный
SSL	Let's Encrypt	Let's Encrypt

4.2. docker-compose.prod.yml

```
services:
  app:
    build:
      context: .
      dockerfile: docker/php/Dockerfile.prod
    container_name: entropiq-prod-app
    restart: always
    ports:
      - "8080:80"
    env_file:
      - .env
    volumes:
      - app-storage:/var/www/html/storage
    depends_on:
      pgsql:
```

```
    condition: service_healthy
  redis:
    condition: service_healthy
networks:
  - entropiq-prod
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost/up"]
  interval: 30s
  timeout: 5s
  retries: 3
  start_period: 30s
deploy:
  resources:
    limits:
      memory: 512M

pgsql:
  image: postgres:16-alpine
  container_name: entropiq-prod-pgsql
  restart: always
  environment:
    POSTGRES_DB: ${DB_DATABASE}
    POSTGRES_USER: ${DB_USERNAME}
    POSTGRES_PASSWORD: ${DB_PASSWORD}
  volumes:
    - pgsql-data:/var/lib/postgresql/data
  networks:
    - entropiq-prod
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${DB_USERNAME}"]
    interval: 10s
    timeout: 5s
    retries: 5
  deploy:
    resources:
      limits:
```



```
memory: 512M
```

```
redis:
```

```
image: redis:7-alpine
container_name: entropiq-prod-redis
restart: always
command: >
  redis-server
  --appendonly yes
  --maxmemory 128mb
  --maxmemory-policy allkeys-lru
  --requirepass ${REDIS_PASSWORD}
```

```
volumes:
```

```
- redis-data:/data
```

```
networks:
```

```
- entropiq-prod
```

```
healthcheck:
```

```
test: ["CMD", "redis-cli", "-a", "${REDIS_PASSWORD}", "ping"]
interval: 10s
timeout: 5s
retries: 5
```

```
deploy:
```

```
resources:
```

```
limits:
```

```
memory: 192M
```

```
volumes:
```

```
app-storage:
```

```
pgsql-data:
```

```
redis-data:
```

```
networks:
```

```
entropiq-prod:
```

```
driver: bridge
```

4.3. docker/php/Dockerfile.prod

Идентичен Dockerfile.stage со следующими отличиями:

```
# В Dockerfile.prod вместо stage.ini:
COPY docker/php/conf.d/php-prod.ini
/usr/local/etc/php/conf.d/custom.ini

# Nginx конфиг для production:
COPY docker/nginx/prod.conf /etc/nginx/http.d/default.conf
```

docker/php/conf.d/php-prod.ini

```
; Opcache (агрессивные настройки)
opcache.enable=1
opcache.memory_consumption=256
opcache.interned_strings_buffer=32
opcache.max_accelerated_files=20000
opcache.validate_timestamps=0
opcache.save_comments=1
opcache.jit=1255
opcache.jit_buffer_size=128M

; Limits
upload_max_filesize = 20M
post_max_size = 25M
memory_limit = 256M
max_execution_time = 30

; Timezone
date.timezone = Europe/Moscow

; Error handling (production)
display_errors = Off
display_startup_errors = Off
log_errors = On
error_log = /var/www/html/storage/logs/php-errors.log
```

```
error_reporting = E_ALL & ~E_DEPRECATED & ~E_STRICT

; Security
expose_php = Off
session.cookie_httponly = 1
session.cookie_secure = 1
session.use_strict_mode = 1
```

4.4. Переменные окружения Production

```
APP_NAME=Entropiq
APP_ENV=production
APP_KEY=base64:...
APP_DEBUG=false
APP_URL=https://entropiq.ru # заменить на актуальный домен

LOG_CHANNEL=stack
LOG_LEVEL=error

DB_CONNECTION=pgsql
DB_HOST=pgsql
DB_PORT=5432
DB_DATABASE=entropiq
DB_USERNAME=entropiq
DB_PASSWORD=<VERY_STRONG_PASSWORD>

REDIS_HOST=redis
REDIS_PASSWORD=<REDIS_PASSWORD>
REDIS_PORT=6379

CACHE_STORE=redis
SESSION_DRIVER=redis
QUEUE_CONNECTION=redis

# Telescope отключён
TELESCOPE_ENABLED=false
```

```
# Sentry
SENTRY_LARAVEL_DSN=https://<key>@sentry.io/<project_id>

# Почта (production SMTP)
MAIL_MAILER=smtp
MAIL_HOST=<smtp_host>
MAIL_PORT=587
MAIL_USERNAME=<username>
MAIL_PASSWORD=<password>
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS="noreply@entropiq.ru"
MAIL_FROM_NAME="${APP_NAME}"

# SMS Aero (боевой режим)
SMSAERO_EMAIL=<email>
SMSAERO_API_KEY=<key>
SMSAERO_SIGN=Entropiq
SMSAERO_TEST_MODE=false

# LLM
GIGACHAT_CLIENT_ID=<client_id>
GIGACHAT_CLIENT_SECRET=<secret>
```

4.5. Скрипт бэкапа БД

Создать `/var/www/entropiq-prod/scripts/backup-db.sh` :

```
#!/bin/bash

set -euo pipefail

BACKUP_DIR="/var/backups/entropiq"
CONTAINER_NAME="entropiq-prod-pgsql"
DB_NAME="${DB_DATABASE:-entropiq}"
DB_USER="${DB_USERNAME:-entropiq}"
```

```

RETENTION_DAYS=7
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="${BACKUP_DIR}/entropiq_${TIMESTAMP}.sql.gz"

mkdir -p "${BACKUP_DIR}"

docker exec "${CONTAINER_NAME}" pg_dump -U "${DB_USER}" "${DB_NAME}" |
gzip > "${BACKUP_FILE}"

if [ -f "${BACKUP_FILE}" ] && [ -s "${BACKUP_FILE}" ]; then
    echo "[$(date)] Backup created: ${BACKUP_FILE} ($(du -h
"${BACKUP_FILE}" | cut -f1))"
else
    echo "[$(date)] ERROR: Backup failed or empty" >&2
    exit 1
fi

find "${BACKUP_DIR}" -name "entropiq*.sql.gz" -mtime
+${RETENTION_DAYS} -delete

echo "[$(date)] Old backups cleaned (retention: ${RETENTION_DAYS}
days)"

```

Добавить в cron:

```

chmod +x /var/www/entropiq-prod/scripts/backup-db.sh

# Открыть crontab
crontab -e

# Добавить (каждые 6 часов):
0 */6 * * * /var/www/entropiq-prod/scripts/backup-db.sh >>
/var/log/entropiq-backup.log 2>&1

```

5. CI/CD — GitHub Actions

5.1. Workflow файл `.github/workflows/ci.yml`

```
name: CI

on:
  pull_request:
    branches: [stage, main]

jobs:
  lint:
    name: Code Style (Pint)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup PHP
        uses: shivammathur/setup-php@v2
        with:
          php-version: '8.3'
          tools: composer:v2

      - name: Install dependencies
        run: composer install --no-interaction --prefer-dist

      - name: Run Pint
        run: ./vendor/bin/pint --test

  analyse:
    name: Static Analysis (Larastan)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup PHP
```

```
uses: shivammathur/setup-php@v2
```

```
with:
```

```
  php-version: '8.3'
```

```
  tools: composer:v2
```

```
- name: Install dependencies
```

```
  run: composer install --no-interaction --prefer-dist
```

```
- name: Run PHPStan
```

```
  run: ./vendor/bin/phpstan analyse --memory-limit=512M
```

```
test:
```

```
  name: Tests (Pest)
```

```
  runs-on: ubuntu-latest
```

```
services:
```

```
  pgsql:
```

```
    image: postgres:16-alpine
```

```
    env:
```

```
      POSTGRES_DB: entropiq_test
```

```
      POSTGRES_USER: test
```

```
      POSTGRES_PASSWORD: test
```

```
    ports:
```

```
      - 5432:5432
```

```
    options: >-
```

```
      --health-cmd="pg_isready -U test"
```

```
      --health-interval=10s
```

```
      --health-timeout=5s
```

```
      --health-retries=5
```

```
  redis:
```

```
    image: redis:7-alpine
```

```
    ports:
```

```
      - 6379:6379
```

```
    options: >-
```

```
      --health-cmd="redis-cli ping"
```

```
--health-interval=10s
--health-timeout=5s
--health-retries=5
```

steps:

- uses: actions/checkout@v4
- name: Setup PHP
uses: shivammathur/setup-php@v2
with:
 php-version: '8.3'
 extensions: pdo_pgsql, pgsql, redis, zip, gd, intl, bcmath,
mbstring
 tools: composer:v2
- name: Install dependencies
run: composer install --no-interaction --prefer-dist
- name: Prepare environment
run: |
 cp .env.example .env
 php artisan key:generate
- name: Run migrations
env:
 DB_CONNECTION: pgsql
 DB_HOST: 127.0.0.1
 DB_PORT: 5432
 DB_DATABASE: entropiq_test
 DB_USERNAME: test
 DB_PASSWORD: test
 REDIS_HOST: 127.0.0.1
run: php artisan migrate --force
- name: Run Pest
env:


```
DB_CONNECTION: pgsql
DB_HOST: 127.0.0.1
DB_PORT: 5432
DB_DATABASE: entropiq_test
DB_USERNAME: test
DB_PASSWORD: test
REDIS_HOST: 127.0.0.1
run: ./vendor/bin/pest --parallel
```

5.2. Автодеплой на Stage (GitHub Actions)

Создать `.github/workflows/deploy-stage.yml` :

```
name: Deploy to Stage

on:
  push:
    branches: [stage]

jobs:
  deploy:
    name: Deploy to Stage Server
    runs-on: ubuntu-latest
    environment: stage

    steps:
      - name: Deploy via SSH
        uses: appleboy/ssh-action@v1
        with:
          host: ${ secrets.STAGE_HOST }
          username: ${ secrets.STAGE_USER }
          key: ${ secrets.STAGE_SSH_KEY }
          script: |
            cd /var/www/entropiq-stage
            git pull origin stage
            docker compose -f docker-compose.stage.yml up -d --build
```

```
docker compose -f docker-compose.stage.yml exec -T app php
artisan migrate --force

docker compose -f docker-compose.stage.yml exec -T app php
artisan config:cache

docker compose -f docker-compose.stage.yml exec -T app php
artisan route:cache

docker compose -f docker-compose.stage.yml exec -T app php
artisan view:cache

docker compose -f docker-compose.stage.yml exec -T app php
artisan event:cache

echo "Stage deployment complete"
```

Необходимые GitHub Secrets (Settings → Secrets and variables → Actions):

Secret	Описание
STAGE_HOST	IP-адрес VPS
STAGE_USER	SSH-пользователь
STAGE_SSH_KEY	Приватный SSH-ключ для доступа к VPS

6. CD — Laravel Envoy

6.1. Что такое Envoy

Laravel Envoy — task runner для удалённых серверов. Файл `Envoy.blade.php` в корне проекта описывает задачи деплоя в Blade-подобном синтаксисе. Envoy подключается к серверу по SSH и выполняет команды.

6.2. Установка

```
# На локальной машине (вне контейнера)
composer global require laravel/envoy
```

Либо через Sail:

```
sail composer require laravel/envoy --dev
```

6.3. Файл `Envoy.blade.php`

Создать в корне проекта:

```
@servers(['stage' => 'deployer@<STAGE_IP>', 'production' =>
'deployer@<PROD_IP>'])

@setup
    $repository = 'git@github.com:<username>/entropiq.git';
    $baseDir = '/var/www';

    if ($on === 'stage') {
        $appDir = $baseDir . '/entropiq-stage';
        $branch = 'stage';
        $composeFile = 'docker-compose.stage.yml';
    } elseif ($on === 'production') {
        $appDir = $baseDir . '/entropiq-prod';
        $branch = 'main';
        $composeFile = 'docker-compose.prod.yml';
    }
@endsetup

@story('deploy', ['on' => $on])
    pull
    build
    migrate
    cache
    health
@endstory

@task('pull')
    echo "Pulling latest changes..."
    cd {{ $appDir }}
```

```
    git pull origin {{ $branch }}
@endtask

@task('build')
    echo "Building and restarting containers..."
    cd {{ $appDir }}
    docker compose -f {{ $composeFile }} up -d --build
@endtask

@task('migrate')
    echo "Running migrations..."
    cd {{ $appDir }}
    docker compose -f {{ $composeFile }} exec -T app php artisan
migrate --force
@endtask

@task('cache')
    echo "Clearing and warming caches..."
    cd {{ $appDir }}
    docker compose -f {{ $composeFile }} exec -T app php artisan
config:cache
    docker compose -f {{ $composeFile }} exec -T app php artisan
route:cache
    docker compose -f {{ $composeFile }} exec -T app php artisan
view:cache
    docker compose -f {{ $composeFile }} exec -T app php artisan
event:cache
@endtask

@task('health')
    echo "Running health check..."
    cd {{ $appDir }}
    sleep 5
    docker compose -f {{ $composeFile }} ps
    echo ""
    echo "Checking HTTP response..."
```

```

    curl -sf http://localhost:8080/up && echo " OK" || echo " FAILED"
@endtask

@task('rollback')
    echo "Rolling back last migration..."
    cd {{ $appDir }}
    docker compose -f {{ $composeFile }} exec -T app php artisan
migrate:rollback --force
@endtask

@task('logs')
    echo "Showing app logs..."
    cd {{ $appDir }}
    docker compose -f {{ $composeFile }} logs --tail=100 app
@endtask

@task('horizon-restart')
    echo "Restarting Horizon..."
    cd {{ $appDir }}
    docker compose -f {{ $composeFile }} exec -T app php artisan
horizon:terminate
@endtask

@finished
    echo "Deployment finished at $(date)"
@endfinished

```

6.4. Использование Envoy

```

# Деплой на stage
envoy run deploy --on=stage

# Деплой на production
envoy run deploy --on=production

# Откат миграции на stage

```

```
envoy run rollback --on=stage

# Просмотр логов на production
envoy run logs --on=production

# Перезапуск Horizon на stage
envoy run horizon-restart --on=stage
```

7. Полезные команды

7.1. Docker

```
# Просмотр запущенных контейнеров
docker compose -f <compose-file> ps

# Логи конкретного сервиса
docker compose -f <compose-file> logs -f app

# Shell внутри контейнера
docker compose -f <compose-file> exec app sh

# Полная очистка (удаление контейнеров, volumes, образов)
docker compose -f <compose-file> down -v --rmi all

# Просмотр использования ресурсов
docker stats
```

7.2. PostgreSQL

```
# Подключение к БД (local через Sail)
```

```
sail psql
```

```
# Подключение к БД (stage/prod)
```

```
docker compose -f <compose-file> exec pgsql psql -U entropiq -d  
entropiq
```

```
# Ручной бэкап
```

```
docker compose -f <compose-file> exec pgsql pg_dump -U entropiq  
entropiq > backup.sql
```

```
# Восстановление из бэкапа
```

```
cat backup.sql | docker compose -f <compose-file> exec -T pgsql psql -  
U entropiq -d entropiq
```

7.3. Redis

```
# Подключение к Redis (local через Sail)
```

```
sail redis
```

```
# Мониторинг команд в реальном времени
```

```
sail redis monitor
```

```
# Очистка кэша Redis
```

```
sail redis flushall
```

7.4. Git Workflow

```
# Создание feature-ветки
```

```
git checkout stage
```

```
git pull origin stage
```

```
git checkout -b feature/CAT-123-add-transactions
```

```
# Коммит изменений
```

```
git add .
```

